

Incident Investigator

A guarded, multi-agent system that investigates manufacturing incidents — production drops, machine downtime, quality defects — against real PostgreSQL and Qdrant data, with a deterministic security layer in front of every question and a live view of exactly what each agent did.

Claude Agent SDK

FastAPI

PostgreSQL

Qdrant

fastembed

React + Vite

WebSocket

00 Overview

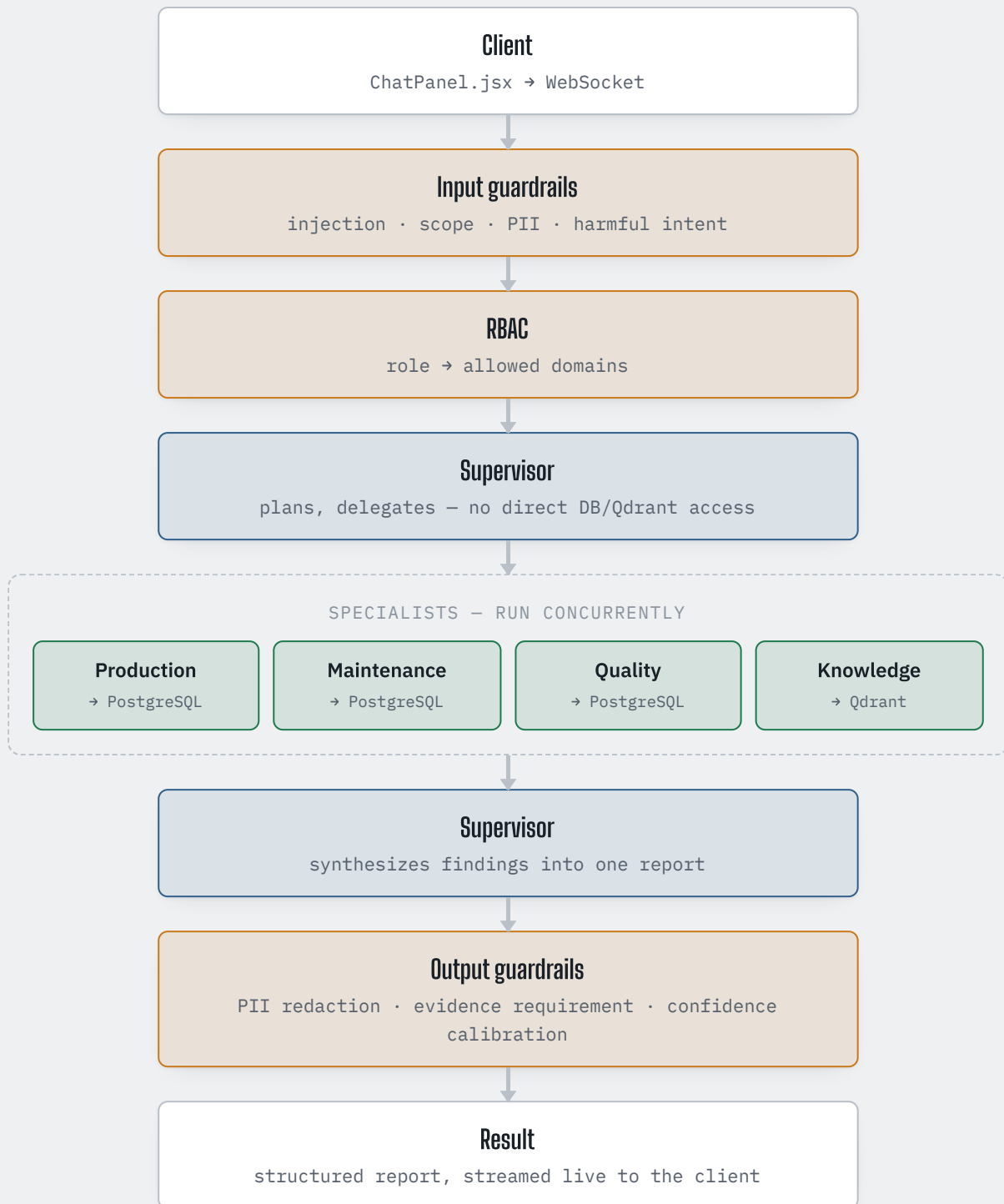
An engineer asks a plain-language question — *“Why did Line 4 production drop yesterday?”* — and a Supervisor agent decides which specialists can actually answer it, runs them concurrently against real production data, and returns one synthesized root-cause report with evidence attached to every claim.

Four specialist agents each own one narrow domain and one set of database tools: **Production**, **Maintenance**, and **Quality** read PostgreSQL; **Knowledge** searches a Qdrant vector store of maintenance manuals and SOPs. None of them can see each other's data, and the Supervisor itself has no direct database access at all — it can only delegate, and every delegation passes through a permission check before any specialist runs.

Two things are true of every answer this system gives: it is backed by numbers a tool actually returned, and it passed through the same input and output checks regardless of how the question was phrased.

01 Request lifecycle

Every question takes the same nine-stage path. The first two stages are plain Python — no model call happens until a question has already cleared them.



The Supervisor calls specialists through exactly one tool, `delegate_to_specialists`, which takes an *array* of requests and fans them out with `asyncio.gather()`. That's a deliberate design choice: an earlier version gave it one delegation tool per specialist, and the model reliably called them one at a time across separate turns even when told it could batch them — collapsing delegation into a single array-input tool removed that failure mode structurally rather than hoping a prompt would fix it.

A second delegation round is sometimes needed — the Supervisor will send the Knowledge Agent a targeted query only *after* Maintenance has told it what actually failed, rather than guessing a search term up front. That's the one place the pipeline is intentionally sequential, because grounding the search in a real finding beats speed here.

02 The specialists

Each agent gets its own Claude Agent SDK session — its own system prompt, its own registered tools, nothing else. There is no shared session for one specialist's tools to leak into another's.

Production

POSTGRESQL

Planned vs. actual quantity, downtime minutes, and per-shift/per-machine breakdown for a line and date.

```
get_production_metrics(line_id, date)
```

```
get_production_metrics_batch(requests[]) —  
batched
```

Maintenance

POSTGRESQL

Machine downtime, failure events, and recurrence history — per machine or per line.

```
get_machine_downtime(machine_id, date)
```

```
get_line_downtime(line_id, date) — every  
machine on a line, one call
```

```
get_line_downtime_batch(requests[]) —  
batched
```

```
get_maintenance_events ·  
get_machine_history
```

Quality

POSTGRESQL

Rejection rate, dominant defect types, and whether today's quality is worse than the line's own history.

```
get_quality_metrics(line_id, date)
```

```
get_quality_metrics_batch(requests[]) —  
batched
```

```
get_defect_distribution ·  
compare_quality_history
```

Knowledge

QDRANT

Semantic search over maintenance manuals, SOPs, and quality procedures — reference knowledge, not live data.

```
search_manufacturing_knowledge(queries[],  
top_k) — batched
```

Every tool above the batch line takes one line/date/machine at a time. That's fine when a question already names a specific date — but a question like *“why did rejection increase?”*, with no date given, forces a specialist to check several candidate dates to find the one that moved. Without a batch option, that becomes N sequential tool calls, each its own full reasoning round trip. The batch tools accept an array of `{line_id, date}` pairs and fan the underlying database reads out concurrently (bounded by a semaphore, not fire-everything-at-once) — collapsing what was a multi-turn scan into one call.

03 Knowledge base

Four reference documents — a maintenance manual, a motor-failure SOP, a quality procedure, and a Line 4 operating procedure — are chunked by section, embedded locally with `fastembed`'s `BAAI/bge-small-en-v1.5` model (384 dimensions, ONNX runtime, no external API call), and stored in Qdrant.

At query time the Knowledge Agent's one tool takes an *array* of queries rather than a single string — the same "one tool, batched input" pattern used everywhere else in this system. Before searching, the agent is instructed to enumerate every distinct angle a question needs (the failure mode, its causes, the diagnostic steps, the repair procedure, the escalation policy) and put all of them in the same call, which then run concurrently against Qdrant. Each result carries its document name, section, matched text, and similarity score — enough for the agent's finding to cite a specific document, and enough for the output guardrail to check that a claim is actually traceable to something retrieved.

04 Security & governance

Every security decision in this system is plain Python — regex, keyword matching, table lookups. Claude is never asked whether a request is safe, in-scope, or well-evidenced; if it were, an adversarial prompt could talk its way past the check.

INPUT GUARDRAILS — RUN BEFORE THE SUPERVISOR EXISTS

Prompt injection Pattern match against known override phrasings ("ignore your instructions", "you are now...").	Scope Keyword-classified against the four domains; off-topic questions with no domain signal are denied.
PII Regex for email, SSN, credit card, phone in the question itself.	Harmful intent Destructive verb + data-target noun combinations ("delete the maintenance records") are denied outright.

RBAC — ROLE → ALLOWED DOMAINS

ROLE	ALLOWED DOMAINS
plant_engineer	production, maintenance, quality, knowledge
quality_auditor	quality, knowledge
maintenance_technician	maintenance, knowledge
guest	knowledge

CAPABILITY TABLE — AGENT → OWNED TOOLS

AGENT	MAY CALL
production	get_production_metrics, get_production_metrics_batch
maintenance	get_machine_downtime, get_line_downtime, get_line_downtime_batch, get_maintenance_events, get_machine_history
quality	get_quality_metrics, get_quality_metrics_batch, get_defect_distribution, compare_quality_history
knowledge	search_manufacturing_knowledge
supervisor	<i>none</i> — delegation only, gated separately

A denial here is a plain dict lookup, not a judgment call — asking Maintenance to answer a production question is refused the same way every time, and the refusal is recorded before the specialist ever runs.

OUTPUT GUARDRAILS — BEFORE A REPORT REACHES THE USER

PII redaction Every text field of the final report is scanned; matches are redacted, not silently passed through.	Evidence requirement A stated root cause with an empty evidence list is flagged — every claim needs at least one supporting fact.
---	---

Confidence calibration

High confidence (≥ 0.9) paired with thin evidence (< 3 items) is flagged as a structural proxy for an unsupported claim.

Every decision from every layer — allow, deny, warn — is written to an in-memory audit log with a timestamp and the investigation ID that produced it, visible to admin accounts from the UI's own audit log page.

05 Frontend

A React + Vite client talks to the FastAPI backend over a single WebSocket (`/ws/investigate`), which streams every pipeline event — guardrail decisions, agent selection, tool calls and results, findings, the final report — as it happens, not just the finished answer.

Summary page

The post-login landing screen: a static, always-green preview of the nine-stage pipeline, plus five example questions (one per specialist, one broad) that prefill the chat box.

Chat + live workflow

A chat panel next to a collapsible sidebar that renders the pipeline diagram *live* — the same stage boxes as the summary page, but wired to real WebSocket events, so a node lights up the instant its agent starts working.

Detail flow page

Every question asked this session, each with its own full trace diagram and step-by-step log — browsable after the fact, not just while it's running.

Audit log page

Admin-only view over the deterministic guardrail and capability decision log described in §04 — every allow, deny, and warn, in order.

06 Performance engineering

A broad question originally took about 4.5 minutes. It now takes 20 seconds to just over a minute, depending on how many specialists and delegation rounds it genuinely needs — measured, not estimated, against the real pipeline and real Postgres/Qdrant data.

WHAT WAS ACTUALLY SLOW

- ➔ **Every session defaulted to the largest available model.**
Nothing in the codebase ever set `model=` on `ClaudeAgentOptions`, so all five sessions — four specialists and the Supervisor — silently ran on `claude-opus-5[1m]`, the heaviest tier, for what is mostly straightforward tool selection and short summarization.
- ➔ **Every session paid for a tool search it didn't need.**
None of the sessions set `tools=`, so Claude Code's full built-in tool preset (Bash, Read, Grep, dozens more) stayed nominally present even though a hard permission layer blocked all of it — which was enough to push every turn into a hidden `ToolSearch` round trip before the real tool call. On a fast model, this sometimes looped until the turn budget ran out and the call failed outright.
- ➔ **Production, Quality, and Maintenance had no batch tool.**
An unspecified-date question ("why did rejection increase?") made a specialist check candidate dates one at a time — 5 to 7 sequential calls observed live, each its own reasoning round trip.
- ➔ **The embedding model loaded lazily.**
The first knowledge search in a fresh process paid a one-time ~25s cost to load the fastembed ONNX model — landing on whichever user happened to ask first.

WHAT CHANGED

BEFORE

Guardrails + RBAC

< 1s

Specialists (concurrent)

≈ 65s

Knowledge follow-up

≈ 99s

Final synthesis

≈ 68s

Total

≈ 232s

➔

AFTER

Guardrails + RBAC

< 1s

Specialists (concurrent)

≈ 39s

Knowledge follow-up

≈ 31s (when needed)

Final synthesis

≈ 17s

Total

58–88s

Model tiering (Haiku for the four specialists, Sonnet for the Supervisor's synthesis) plus `tools=` on every session plus the three new batch tools plus moving the embedding-model load to server startup. No prompt was loosened and no guardrail, RBAC check, or evidence requirement was touched — the same 133-test suite passes before and after.

LIVE, END-TO-END — REAL QUESTIONS, REAL DATA

Production quantity (narrow)	≈ 23s
Broad multi-agent root cause	≈ 55–63s
Machine downtime + SOP follow-up	≈ 69s
Knowledge-only lookup	≈ 44s
Quality, no date given (multi-week scan)	≈ 112s

07 Stack reference

LAYER	USED FOR
Claude Agent SDK (Python)	Every agent session — system prompt, registered tools, model choice, turn limits
Claude Haiku 4.5	Production, Maintenance, Quality, Knowledge — tool selection + short summarization
Claude Sonnet	Supervisor — cross-specialist synthesis, root cause vs. contributing factors
FastAPI + uvicorn	HTTP/WebSocket backend, session auth, audit-log endpoint
PostgreSQL + SQLAlchemy	Production, maintenance, and quality tables — the real, computed source of every number
Qdrant	Vector store for the manufacturing knowledge base
fastembed	Local embedding model (BAAI/bge-small-en-v1.5, 384-dim), no external API call
React 19 + Vite	Summary page, chat panel, live workflow trace, detail flow, audit log
JWT session cookies	Login/role state — plant_engineer, quality_auditor, maintenance_technician, guest, admin